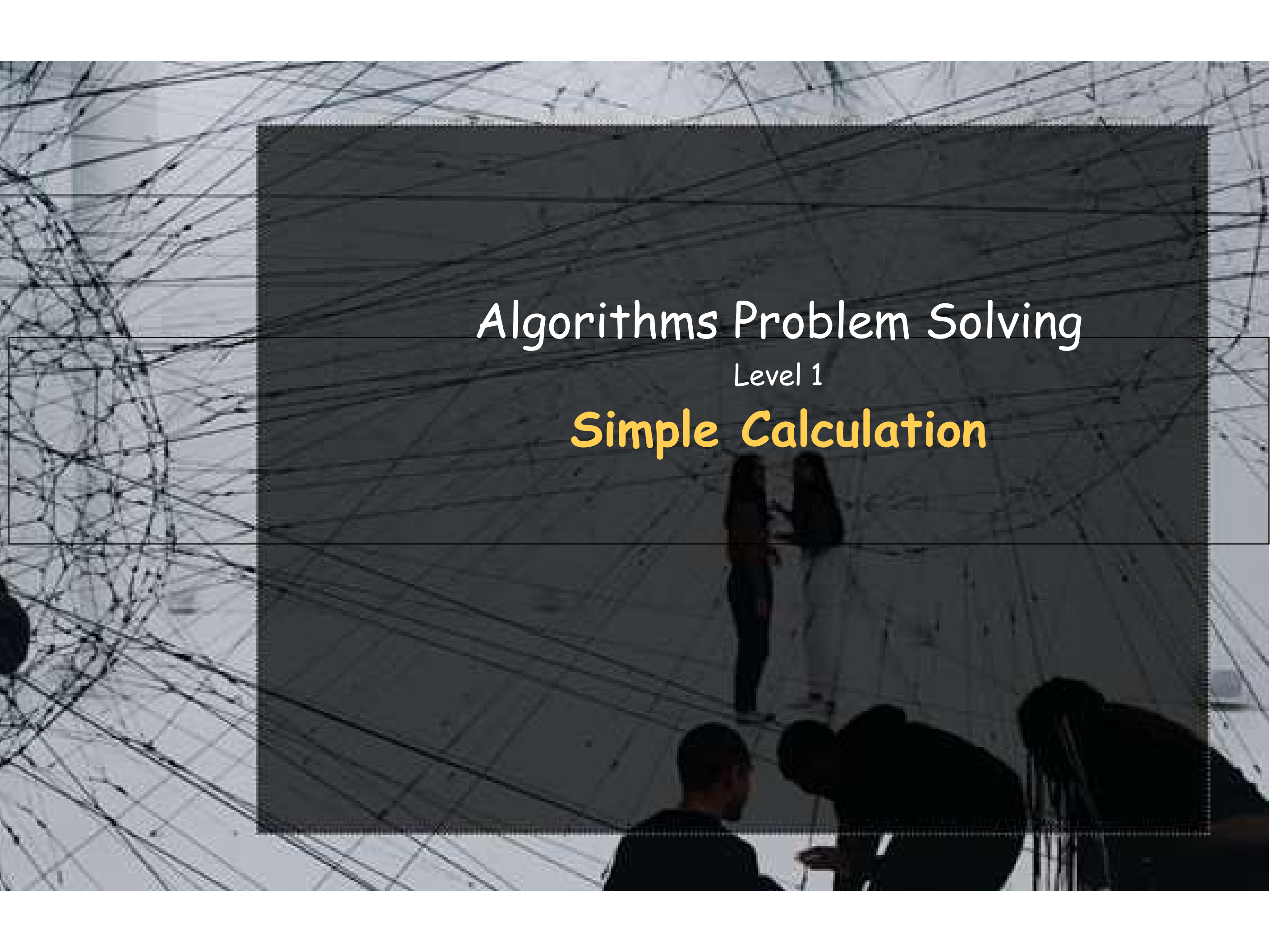


Algorithms Problem Solving

Level 1

Simple Calculation



Algorithms Problem Solving

Level 1

Simple Calculation

Definition

- ◆ This program asks the user to enter two numbers and an operation, then performs the selected arithmetic calculation.
- ◆ The supported operations are addition, subtraction, multiplication, and division.
- ◆ The program uses an `enum`, a `struct`, a `switch` statement, and functions to organize the calculator.

Explanation

- ◆ ``enOperation`` is an enumeration that represents the four supported operations:
 - ``ADD = '+'``
 - ``SUBTRACT = '-'``
 - ``MULTIPLY = '*'``
 - ``DIVIDE = '/'``
- ◆ ``stCalculatorInput`` is a structure that groups the calculator inputs:
 - ``Number1`` stores the first number.
 - ``Number2`` stores the second number.
 - ``Operation`` stores the selected operation.
- ◆ ``ShowOperationsMenu()`` displays the available operation symbols.
- ◆ ``ReadNumber()`` reads a number using a message passed as a ``string`` parameter.
- ◆ ``ReadOperation()`` reads a character and converts it to ``enOperation``.
- ◆ ``ReadCalculatorInput()`` collects both numbers and the operation and returns them as one ``stCalculatorInput`` object.
- ◆ ``Calculate()`` uses a ``switch`` statement to determine which arithmetic operation should be performed.
- ◆ For division, the program checks ``num2 != 0`` before dividing to avoid division by zero.
- ◆ ``PrintResult()`` displays the calculated result.
- ◆ In ``main()``, the input structure is created first, then its members are passed to ``Calculate()``.

Steps in main()

1. Read the first number.
2. Read the second number.
3. Display the available operations.
4. Read the selected operation.
5. Store the inputs in `stCalculatorInput`.
6. Pass the numbers and operation to `Calculate()`.
7. Use `switch` to select the correct calculation.
8. Return the result.
9. Print the result.

💡 Example

◆ If the user enters:

Number 1: 10

Number 2: 5

Operation: *

◆ The program selects the `MULTIPLY` case.

◆ Calculation:

$$10 \times 5 = 50$$

➡ Result

Result = 50

✚ Note

◆ The enum values are assigned directly to the operation symbols:

``ADD = '+'``

``SUBTRACT = '-'``

``MULTIPLY = '*'``

``DIVIDE = '/'``

◆ This allows the character entered by the user to be converted into the corresponding enum value:

``enOperation(op)``

◆ The ``switch`` statement can then use ``ADD``, ``SUBTRACT``, ``MULTIPLY``, and ``DIVIDE``.

⊘ Warning

- ◆ Division by zero is checked explicitly.
- ◆ If the second number is `0`, the program prints an error instead of performing the division.
- ◆ Invalid operation symbols are also handled by the `default` case.



Comparison

▨ `switch`

- ✓ Checks the value of `op`.
- ✓ Each `case` represents one supported operation.
- ✓ Suitable when there are several fixed choices.

▤ `if-else`

- ✓ Could also compare the operation symbol.
- ✓ Useful for conditions involving ranges or more complex expressions.
- ✓ In this program, `switch` makes the fixed operation choices clear.

✦✦ Summary

- ✓ `enum` can represent a fixed set of operations.
- ✓ `struct` can group different types of calculator input.
- ✓ `switch` is useful for selecting between fixed choices.
- ✓ Division by zero should be checked before performing division.
- ✓ Functions separate input, calculation, and output.
- ✓ Function return values can be passed directly into another function.

Thank you 😊